



**Pontificia Universidad Católica de Valparaíso**

**Facultad de Ingeniería**

**Escuela de Ingeniería Informática**

# **Telemedicina**

**Autor:** Francisca Zamora

**Asignatura:** Ingeniería web y móvil

**Profesor:** Gabriel Toro

# Introducción

El presente proyecto surge en respuesta a la creciente necesidad de modernizar los canales de atención de salud mediante herramientas digitales avanzadas. La telemedicina no solo representa una alternativa a la consulta presencial, sino que se posiciona como una infraestructura crítica para el seguimiento continuo de pacientes. Por eso, se desarrolla una aplicación multiplataforma utilizando el framework **ionic con React**, diseñada para facilitar el intercambio de información clínica en tiempo real. El objetivo principal es transformar la relación médico-paciente de un modelo reactivo (atención solo ante la crisis) a un modelo proactivo y preventivo, centrado en el monitoreo sistemático y la comunicación eficiente.

## Entrega parcial 1:

### 1. Justificación del problema y análisis del usuario objetivo. (EP 1.2)

#### Planteamiento del Problema

La problemática central identificada es la discontinuidad de la información médica una vez que el paciente abandona el centro asistencial o bien termina su videollamada. Actualmente, los pacientes con enfermedades crónicas enfrentan dificultades para registrar cambios diarios en su condición, ya que si se encuentran mal o no tienen bien claro si mejoraron, deben ir a una consulta presencial, pero esto genera diagnósticos basados en datos incompletos y subjetivos. Esta falta de visibilidad clínica entre consultas impide que el profesional de la salud pueda intervenir de manera oportuna ante variaciones críticas de signos vitales, derivando en complicaciones que frecuentemente terminan en servicios de urgencia saturados. Además, se busca que la conectividad entre médico-paciente sea más sencilla a través de la casa.

#### Justificación del problema

La implementación de esta solución tecnológica se justifica por su capacidad de centralizar el flujo de datos clínicos en un entorno seguro y accesible. Al integrar herramientas de autoregistro de signos vitales y un canal de comunicación directo, la plataforma permite:

- **Optimización de Tiempos:** Reduce la necesidad de traslados para consultas que pueden resolverse de forma remota.
- **Toma de Decisiones Basada en Datos:** Proporciona al médico un historial gráfico y detallado de la evolución del paciente.
- **Prevención y Alerta:** Permite la detección temprana de riesgos mediante un sistema de semaforización visual según la gravedad de los registros.

#### Análisis del usuario objetivo

La plataforma ha sido diseñada bajo principios de **Diseño Centrado en el Usuario (UCD)**, reconociendo dos perfiles con necesidades técnicas diferenciadas:

- **Perfil Paciente:** Personas que requieren monitoreo constante, con una interfaz simplificada para facilitar el registro de datos vitales, incluso en adultos mayores.
- **Perfil Médico:** Profesionales que necesitan gestionar múltiples pacientes, visualizar datos históricos y priorizar casos críticos mediante alertas automáticas.

## 2. Definición de Requerimientos (EP 1.1)

### Requerimientos Funcionales

Para esta etapa, se han definido y diseñado las interfaces para los siguientes 9 requerimientos funcionales:

1. **RF1 - Registro de Estado Diario:** El paciente puede ingresar signos vitales y síntomas.
2. **RF2 - Visualización de historial de salud:** El sistema resalta automáticamente el último estado de salud registrado.
3. **RF3 - Gestión de Agenda y Citas:** Interfaz para visualizar citas próximas y pasadas.
4. **RF4 - Comunicación médico-paciente:** El sistema permitirá el intercambio de mensajes entre el médico y el paciente para resolver dudas o entregar indicaciones.
5. **RF5 - Gestión de pacientes:** El médico podrá visualizar y gestionar la lista de pacientes asignados a su atención.
6. **RF6 - Sistema de Alertas Visuales:** Diferenciación por colores (rojo para crítico, verde para estable) según los registros de salud.
7. **RF7 - Ficha Clínica del Paciente (Vista Médico):** Panel detallado para el profesional con antecedentes y evolución gráfica.
8. **RF8 - Emisión de recomendaciones médicas:** El médico podrá enviar recomendaciones o indicaciones al paciente en base a su estado de salud registrado.
9. **RF9- Registro de cumplimiento de tratamiento:** El paciente podrá indicar si ha seguido o no las indicaciones médicas, permitiendo al médico evaluar la adherencia al tratamiento.
10. **RF10 - Registro y Validación de Usuarios:** Formularios de acceso con validación de RUT, Región y Comuna.
11. **RF11 - Notificaciones de Seguimiento:** Indicadores visuales sobre recordatorios de medicamentos o toma de presión.

### Requerimientos No Funcionales (RNF)

1. **RNF1 - Seguridad:** Implementación de rutas protegidas y autenticación basada en estados para garantizar la confidencialidad.
2. **RNF2 - Rendimiento:** Tiempo de carga y respuesta de navegación entre vistas inferior a 2 segundos.
3. **RNF3 - Usabilidad:** Diseño responsivo con componentes de Ionic que facilitan la interacción táctil y lectura clara.
4. **RNF4 – Escalabilidad:** El sistema debe ser capaz de soportar un aumento en la cantidad de usuarios pacientes y médicos, sin afectar significativamente su rendimiento.

### 3. Bocetos de UI/UX y Prototipo en Figma (EP 1.3)

Se han desarrollado mockups funcionales que cubren el flujo completo de la aplicación, diferenciando explícitamente entre las versiones **Desktop** y **Mobile**.

Actualmente, **el prototipo implementado en Figma corresponde al módulo principal de Telemedicina**, permitiendo validar navegación, arquitectura de información y experiencia de usuario del sistema.

Respecto a la integración con el portal institucional de la Municipalidad de Santo Domingo, esta fue definida a nivel conceptual como parte del diseño UX, proponiendo un **acceso directo desde el sitio oficial Municipalidad de Santo Domingo hacia el módulo de Telemedicina**. Esta integración visual **actualmente no se encuentra prototipada en Figma**, ya que se desarrolló los mockups para entrar a los módulos correspondientes.

Los mockups se encuentran en la carpeta otros, en carpeta zip descargado.

#### Formularios de Acceso y Validación Visual

Se diseñaron dos formularios críticos con un enfoque en **Diseño Centrado en el Usuario (UCD)**:

- **Pantalla de Inicio de Sesión (Login):**
  - **Campos:** RUT y Contraseña.
  - **UX:** Se incluyen estados de validación visual, bordes rojos y mensajes de "Dato Requerido" para prevenir errores de envío. El diseño es limpio, con el logo institucional en alta jerarquía para generar confianza.
- **Pantalla de Registro:**
  - **Campos requeridos:** Nombre y Apellido, RUT, Correo Electrónico, Región, Comuna, Contraseña y Confirmación de Contraseña.
  - **Validaciones:** Incluye el checkbox obligatorio de **Aceptación de Términos y Condiciones**. El diseño utiliza una distribución clara para evitar la fatiga visual ante la cantidad de campos.

#### Pantallas Funcionales (Mockups)

Se diseñaron las siguientes pantallas diferenciadas, cada una asociada a un requerimiento funcional:

1. **Dashboard del Paciente:** Presenta un resumen de salud inmediato, destacando el último estado registrado, las próximas citas médicas y mensaje enviado por médico. Al igual que tiene iconos directos para saber indicaciones, recomendaciones, agendar, etc.
2. **Registro de Estado de Salud:** Formulario intuitivo con selectores para registrar presión arterial, pulso y síntomas.

3. **Módulo de Chat (Paciente):** Interfaz de mensajería para comunicación directa con el médico asignado.
4. **Dashboard del Médico:** Panel de gestión que muestra el resumen de pacientes críticos mediante un sistema de alertas.
5. **Lista de Pacientes Asignados:** Tabla con filtros de búsqueda por nombre o RUT, facilitando la administración de la carga asistencial. Aparecen de igual forma, de los primeros los pacientes en estado crítico.
6. **Ficha Clínica Detallada:** Visualización de antecedentes del paciente y gráficos de evolución de signos vitales o enfermedades que padezca.
7. **Interfaz de Videollamada:** Pantalla con controles de audio/video y vista compartida para la atención remota.

Se utilizó una paleta de colores **Verde Esmeralda (#00875E)** y blancos para transmitir higiene y profesionalismo médico. La jerarquía se establece mediante el peso tipográfico y la ubicación espacial, situando la información de alerta siempre en la parte superior del campo visual.

## 4: Definición de Arquitectura de Navegación y Experiencia del Usuario (EP 1.4)

Se detalla la organización estructural y el flujo de interacción de la plataforma, garantizando una experiencia de usuario coherente.

### (a) Rutas Principales y Secundarias

La aplicación implementa un sistema de enrutamiento basado en **React Router**, dividiendo el acceso en dos niveles jerárquicos:

- **Rutas Principales (Nivel 1):** Son los puntos de acceso públicos.
  - **Login:** Interfaz de autenticación donde el usuario ingresa su RUT y contraseña. El sistema bloquea el acceso si los campos están vacíos mediante validaciones de estado.
  - **Registro:** Formulario de captura de datos, Nombre, RUT, Correo, Región, Comuna y Contraseña necesario para nuevos usuarios.
- **Rutas Secundarias (Nivel 2 Protegidas):** Vistas operativas accesibles únicamente tras una autenticación exitosa.
  - **Home:** Dashboard centralizado.
  - **Chat:** Módulo de comunicación con el especialista.
  - **Registro-estado:** Formulario de entrada de signos vitales.

### (b) Relaciones Jerárquicas entre Vistas

La jerarquía se organiza de forma arbórea desde un nodo raíz (App.tsx).

- **Nivel Superior:** Control de sesión. Si el authService confirma la sesión, se habilita el IonRouterOutlet.

- **Nivel Intermedio:** El **Dashboard** actúa como el eje central. Todas las vistas secundarias como chat y registro de salud dependen jerárquicamente del Home, permitiendo siempre el retorno a la vista principal para mantener el contexto del usuario.

#### (c) Flujo de Navegación entre Funcionalidades

La interacción entre pantallas ha sido diseñada para ser fluida y sin recargas de página (SPA).

- **Desde el Dashboard:** El usuario puede navegar radialmente hacia el registro de salud o la mensajería.
- **Interacción de Datos:** Al completar un registro de salud, el flujo redirige al usuario de vuelta al Dashboard, donde se actualiza el **Resumen de Salud** para reflejar los nuevos datos ingresados, cerrando el ciclo de retroalimentación.

#### (d) Diferenciación de Acceso según Roles

La arquitectura de la aplicación reconoce y adapta la interfaz según el perfil del usuario:

- **Rol Paciente:** Enfocado en el autoregistro. Su navegación permite ver indicaciones médicas, exámenes y agendar citas. El menú lateral prioriza herramientas de seguimiento personal.
- **Rol Médico:** Su navegación es de **supervisión y gestión**. El médico no registra datos propios, sino que accede a una **Lista de Pacientes Asignados**. Puede entrar en la ficha de cada paciente, revisar gráficos de evolución histórica, emitir recomendaciones y activar videollamadas de telemedicina.

#### (e) Flujo de Principales Tareas (Task Flow)

Se han optimizado los flujos para minimizar la cantidad de clics:

1. **Task Flow Paciente:** Login -> Home -> Botón "Registrar estado" -> Ingreso de datos -> Confirmación -> Regreso al Home con resumen actualizado.
2. **Task Flow Médico:** Login -> Lista de Pacientes -> Selección de Ficha -> Revisión de alertas rojas/verdes -> acceso de Chat o Videollamada de respuesta.

#### (f) Puntos Críticos de Interacción

Se identificaron y resolvieron nodos de fricción para mejorar la UX:

- **Validación Bloqueante:** En Login y Registro, el botón de acción no se procesa si faltan datos obligatorios, marcando los campos en rojo para una corrección inmediata.
- **Feedback de Estado:** El uso de semaforización visual en el Dashboard permite al usuario entender su estado de salud de un solo vistazo sin leer texto denso.
- **Cierre de Sesión Seguro:** Se implementó `window.location.assign` para limpiar el historial del navegador, evitando que se pueda volver atrás a una ruta protegida tras cerrar sesión.

#### (g) Coherencia de Experiencia entre Dispositivos

La adaptabilidad se logra mediante el sistema de grillas de Ionic:

- **Vista Web:** El menú lateral es fijo y el contenido se despliega en paneles laterales, aprovechando el espacio horizontal.
- **Vista Móvil:** El diseño se reestructura a una sola columna. Los inputs y botones se expanden para facilitar el uso táctil, manteniendo la jerarquía de la información crítica siempre en la parte superior del scroll.

#### (h) Justificación Técnica

La arquitectura se sustenta en tres pilares:

1. **Usabilidad y Eficiencia:** El uso de componentes nativos de Ionic asegura transiciones suaves y un diseño familiar para el usuario.
2. **Claridad Estructural:** La separación en carpetas facilita el mantenimiento del código.
3. **Escalabilidad:** Al usar **React Router**, la estructura está lista para crecer, donde se integrarán las llamadas reales a la API y la base de datos sin necesidad de reescribir la lógica de navegación.

## 5. Creación del proyecto en Ionic con React (EP 1.5 y 1.6)

### Componentes Utilizados

Para garantizar la coherencia visual y la escalabilidad del frontend, se utilizaron componentes nativos de **Ionic Framework**:

- **IonGrid, IonRow e IonCol:** Implementados para crear una arquitectura responsiva que adapta el Dashboard de una vista extendida en escritorio a una disposición vertical en dispositivos móviles.
- **IonSelect e IonInput:** Utilizados para la captura de datos clínicos, permitiendo validaciones visuales inmediatas (bordes rojos) ante campos obligatorios vacíos.
- **IonIcon:** Integrados para mejorar la jerarquía visual y la navegabilidad intuitiva en los menús y paneles de control.

### Alcance Actual del Proyecto

En esta **entrega**, el software cumple con los siguientes hitos:

1. **Navegación Estructural:** El sistema permite el tránsito fluido entre las 4 pantallas principales implementadas, Login, Registro, Home y Registro de Estado.
2. **Seguridad de Rutas:** Implementación funcional de RutaProtegida. El acceso al contenido interno está bloqueado si no existe una sesión activa.
3. **Gestión de Sesión (Mock):** El sistema simula procesos de Login y Logout, gestionando el estado de la aplicación y redirigiendo al usuario correctamente.
4. **Validación de Formularios:** Los campos de entrada verifican la presencia de datos antes de permitir el avance, proporcionando feedback visual al usuario.



## Limitaciones y Trabajo Pendiente

Se reconocen las siguientes brechas técnicas que serán resueltas en siguientes entregas:

- **Autenticación y Registro Real:** Actualmente, el sistema no valida la veracidad del RUT o la contraseña contra una base de datos; solo verifica que los campos no estén vacíos. Falta implementar la lógica de backend para verificar credenciales reales.
- **Refinamiento de UI Móvil:** Aunque la estructura es responsiva, el diseño actual está optimizado primordialmente para Web. Falta ajustar dimensiones y espaciados específicos para mejorar la experiencia táctil en dispositivos móviles.
- **Interactividad del Menú Lateral:** Los iconos y enlaces del menú de navegación, Citas, Exámenes, Indicaciones, etc. Se encuentran en estado visual (mockup). Su funcionalidad y vinculación a páginas reales están pendientes.
- **Complejidad del Registro Clínico:** El formulario de "Registro de Salud" cuenta con categorías limitadas. Se ampliará el catálogo de síntomas y constantes vitales en la siguiente fase.
- **Módulo de Chat Dinámico:** La mensajería es actualmente una interfaz estática. Falta implementar la lógica para responder mensajes, visualizar historiales pasados y filtrar conversaciones por profesional de salud.
- **Persistencia de Datos:** La información es volátil o se almacena en localStorage. Se requiere la integración de una **API REST (Node.js/Python)** y una **Base de Datos Relacional** para el manejo de información persistente y segura.

## Entrega parcial 2:

### 1. Creación del servidor en Node.js con Express (EP 2.1)

Se implementó un servidor backend utilizando **Node.js** con el framework **Express**, configurado bajo el entorno de **TypeScript**. Esta combinación fue seleccionada debido a que Express proporciona la velocidad y flexibilidad arquitectónica que requiere una aplicación médica, mientras que TypeScript añade un sistema de tipado estático que previene errores de compilación y manipulación en los datos sensibles de salud.

- **Estructura Modular:** En lugar de concentrar el código en un único archivo, el backend se estructuró de manera modular. El servidor principal se limita exclusivamente a inicializar los servicios y los middlewares, delegando la gestión de las rutas (rutas/) y el procesamiento de la lógica de negocio (controladores/) en módulos independientes y ordenados.
- **Middlewares Esenciales:** Se incorporó el middleware nativo `express.json()` para permitir que el servidor interprete de forma automática los cuerpos de las peticiones en formato JSON provenientes del cliente. Asimismo, se configuró el middleware `cors()` con el objetivo de otorgar permisos explícitos al frontend móvil para comunicarse e intercambiar recursos de manera segura con la API REST.

### 2. Configuración y Modelado de la Base de Datos Relacional (EP 2.2)

Para este proyecto se implementó una base de datos relacional con **MySQL** corriendo de forma local en **MySQL Workbench**, utilizando **Prisma ORM** como herramienta de abstracción para gestionar todo el modelo directamente desde el código fuente. Con este enfoque, el sistema opera de manera autónoma y local en el computador del evaluador sin depender de configuraciones externas.

En el archivo `schema.prisma` se definieron dos relaciones clave para estructurar los datos y asegurar que la información no quede duplicada ni huérfana:

- **Relación de Entidad Única (Médico - Paciente):** Se centralizó a todos los usuarios en la entidad `Usuario`, quedando diferenciados únicamente por el campo `rol` (médico o paciente). Un usuario con rol de médico puede tener múltiples pacientes a su cargo mediante el campo `medicoid`. Para esta relación se aplicó la regla `onDelete: SetNull`. Esto garantiza que, si un médico es desvinculado o eliminado del sistema, las cuentas de sus pacientes permanezcan activas e intactas en MySQL.
- **Relación Uno a Muchos (Paciente - Registro Clínico):** Cada ficha de signos vitales guardada en la tabla `RegistroSalud` se vincula con el ID único del paciente a través del campo `usuarioid`. En este caso se configuró la regla `onDelete: Cascade`. Esto significa

que, si un paciente es eliminado permanentemente de la plataforma, todas sus fichas clínicas asociadas se borrarán de forma automática y simultánea en Workbench, evitando registros huérfanos y resguardando la confidencialidad.

La creación física de las tablas no requirió la escritura manual de scripts SQL en Workbench. El proceso se delegó por completo al flujo del ORM mediante el comando `npx prisma db push`. Este comando interpreta las restricciones del esquema, generando de manera automática las tablas, llaves foráneas e integridades dentro de la instancia local de MySQL Workbench.

### 3.Desarrollo de API REST con Endpoints Básicos (EP 2.3)

Se construyó una API REST estructurada para la comunicación entre el cliente móvil y el servidor. Cada petición enviada por la aplicación recibe una respuesta bajo la arquitectura JSON, acompañada de un código de estado HTTP estándar que determina el éxito o fallo de la operación:

- **POST /api/registro:** Procesa la creación de cuentas de usuarios en el sistema. El controlador valida la presencia de los campos obligatorios y verifica que el RUT o el correo electrónico no existan previamente en la base de datos MySQL, retornando un estado 201 Created en caso de éxito.
- **POST /api/login:** Valida las credenciales de acceso mediante el RUT y la contraseña. Si los datos coinciden con el registro de la base de datos, el servidor genera la sesión correspondiente y responde con un estado 200 OK.
- **POST /api/registro-salud:** Recibe los datos biométricos ingresados (presión arterial, pulso, temperatura, nivel de dolor y síntomas). El backend ejecuta una evaluación condicional de los parámetros para determinar de forma matemática si el estado es "Estable" o "Crítico", persistiendo la ficha en MySQL con un estado 201 Created.
- **GET /api/registro-salud/:usuarioid:** Consulta las tablas de la base de datos utilizando el identificador único del paciente para extraer de forma cronológica descendente todo su historial médico y aislar el último control clínico, devolviendo un estado 200 OK.
- **GET /api/medico/:id/dashboard:** Recupera la lista de los pacientes vinculados directamente al ID del médico consultado. El endpoint consolida la información necesaria para el monitoreo y devuelve los registros con un estado 200 OK.
- **PUT / DELETE /api/registro-salud/:id:** Permiten la modificación de las observaciones clínicas o la remoción física de un registro médico específico mediante su identificador único, respondiendo con un estado 200 OK para asegurar el cumplimiento del estándar CRUD en la arquitectura REST.

### 4.Consumo de la API REST desde Ionic (EP 2.4)

El consumo de los servicios web de la API REST se centralizó en el frontend mediante una instancia configurada de **Axios** ubicada en src/api.ts. Esta implementación reemplazó las llamadas HTTP manuales por un cliente unificado que integra un **Interceptor de Peticiones**.

El interceptor evalúa automáticamente cada solicitud saliente dirigida hacia el servidor Express. Si detecta la existencia de una clave válida de autenticación almacenada en el navegador, extrae dicho elemento y lo acopla dinámicamente en la cabecera Authorization de la petición HTTP bajo el esquema estándar Bearer. Esta configuración técnica automatiza la transferencia de las credenciales de sesión en segundo plano, garantizando que el backend reciba y valide la identidad del usuario en cada endpoint protegido sin requerir interrupciones manuales en la interfaz de la aplicación.

## 5. Implementación de Autenticación con JWT (EP 2.5)

La gestión y el control de las sesiones de usuario se estructuraron mediante el estándar **JWT**. Al procesar una autenticación exitosa en el endpoint de inicio de sesión, el backend genera y firma un token criptográfico utilizando una clave secreta definida en las variables de entorno

- **Rutas Protegidas:** En el archivo de enrutamiento del frontend (App.tsx), se implementó el componente especializado <RutaProtegida>. Este componente actúa como un filtro de acceso en el cliente que evalúa la presencia del token antes de renderizar la vista solicitada. Si se intenta ingresar de forma directa a través de la URL a los módulos internos (como /home o /medico-dashboard) sin una sesión activa, el sistema intercepta la navegación y redirige de forma automática al usuario hacia la interfaz de /login.
- **Diferenciación por Roles:** El token emitido por el servidor almacena internamente en su estructura de datos (payload) el rol específico asignado al usuario en la base de datos MySQL. Al resolverse la petición de inicio de sesión, el frontend decodifica dicha información para ejecutar una bifurcación lógica en la navegación: si el registro posee el rol de médico, la aplicación realiza un direccionamiento inmediato hacia la ruta /medico-dashboard, mientras que si el rol corresponde al de un paciente, el flujo deriva el acceso hacia la interfaz de /home

## 6. Validación de Usuarios y Seguridad Avanzada (EP 2.6)

El sistema gestiona la seguridad y consistencia de los accesos mediante políticas de verificación en el servidor y el uso de componentes de abstracción de datos:

- **Manejo Seguro de Credenciales en Tránsito:** Durante el proceso de autenticación e inicio de sesión, el sistema valida la identidad contrastando el RUT y la contraseña en texto plano de manera directa contra los registros existentes en la base de datos MySQL. Esto permite una verificación lineal e inmediata de los datos introducidos por el usuario en la interfaz del cliente móvil antes de conceder el token de acceso.

- **Protección contra Inyección SQL mediante Parametrización Nativa:** Mediante el uso de **Prisma ORM**, todas las variables capturadas desde las entradas de texto del formulario e inyectadas en los métodos del modelo (findFirst, create, findMany) son automáticamente sanitizadas y parametrizadas por el motor del ORM antes de interactuar con MySQL. Esta capa técnica anula el riesgo de ataques por inyección SQL, debido a que las entradas del usuario son tratadas estrictamente como valores de datos y nunca como instrucciones de código SQL ejecutable.
- **Validación de Inputs en el Backend:** Los controladores de Express implementan estructuras de control condicionales que verifican la presencia y el formato correcto de los campos obligatorios (como el RUT, el correo electrónico y la contraseña) antes de dar paso a cualquier operación de escritura o lectura en MySQL, devolviendo códigos de error 400 Bad Request o 401 Unauthorized en caso de detectar datos nulos, incompletos o inconsistentes.

**Limitaciones:** Siguen algunas limitaciones comentadas en la entrega 1. Aunque el backend y la base de datos MySQL funcionan al 100% a través de Postman, la aplicación en Ionic todavía tiene las siguientes limitaciones técnicas que se resolverán en la entrega final:

- **Datos congelados en el inicio:** La pantalla del paciente solo muestra la información que se cargó al momento de hacer el login (/login). Si se registra un nuevo dato, la pantalla no se entera porque sigue leyendo variables fijas del localStorage.
- **Formularios desconectados:** La pantalla para registrar los signos vitales (/registro-estado) muestra los campos y guarda los datos en el backend, pero el botón de envío aún no está amarrado para refrescar la pantalla principal automáticamente. Por ahora, toda la inserción real de datos clínicos se realiza y valida de manera impecable desde Postman.
- **Pantallas en estado de maqueta:** Secciones como el chat, la agenda de citas y el historial de videollamadas son maquetas estáticas. Muestran el diseño visual pero todavía no consumen datos reales del servidor Express.

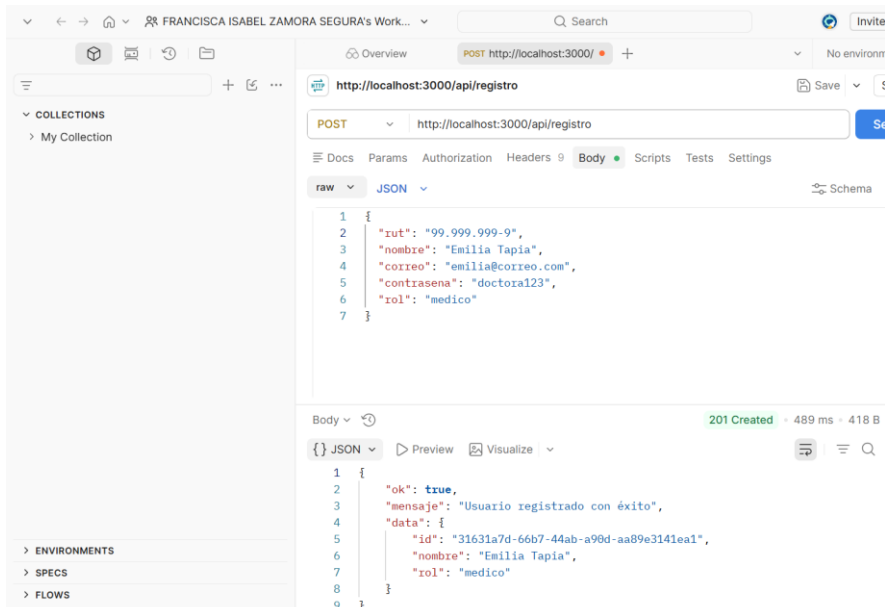
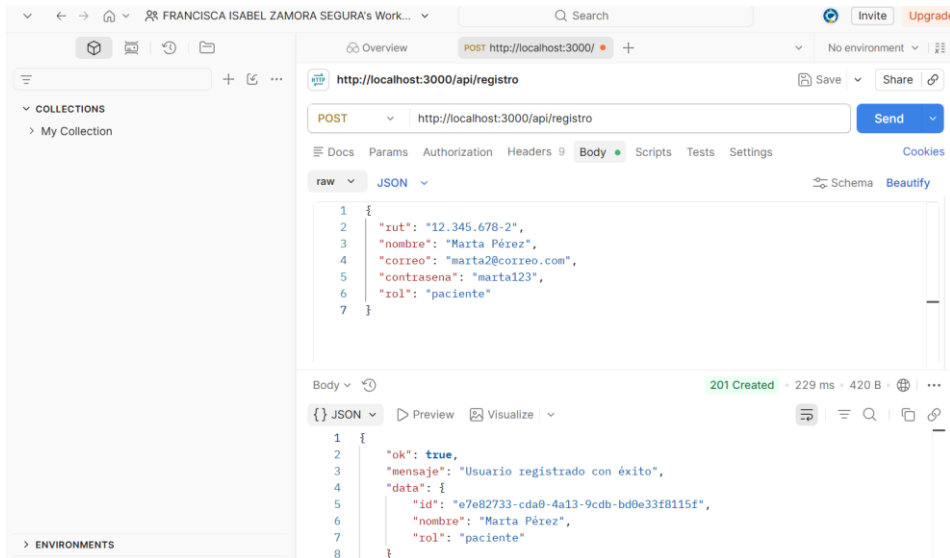
## 7.Pruebas funcionales (EP 2.7)

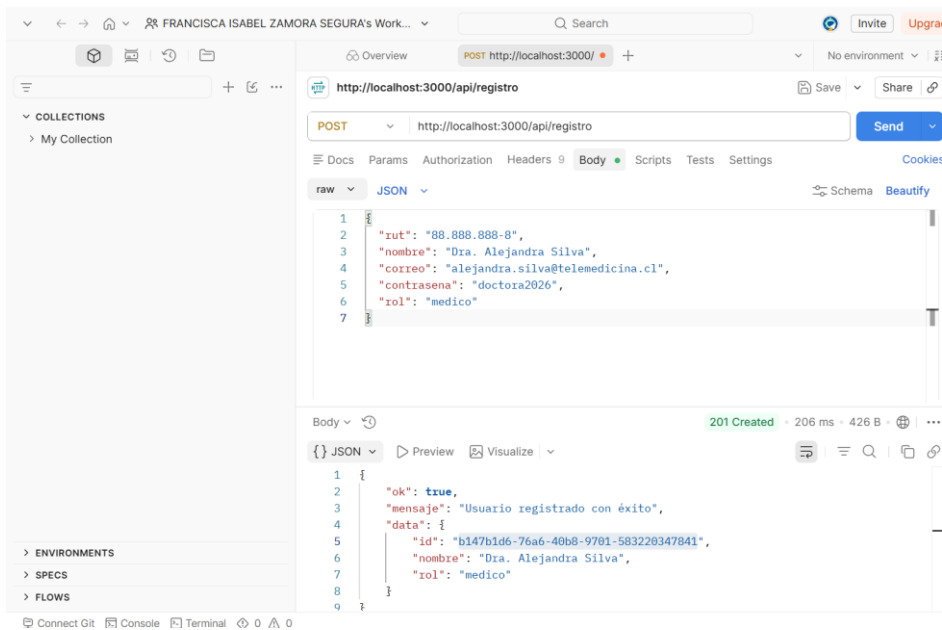
Se realizaron pruebas funcionales estructuradas mediante **Postman**. Los resultados y respuestas del servidor proporcionan la evidencia del comportamiento lógico del backend:

**Registro de Usuarios** Se verificó el flujo de creación de cuentas mediante el envío de solicitudes con payloads JSON estructurados en el cuerpo de la petición. El backend procesa de manera correcta los datos de entrada según los roles del sistema:

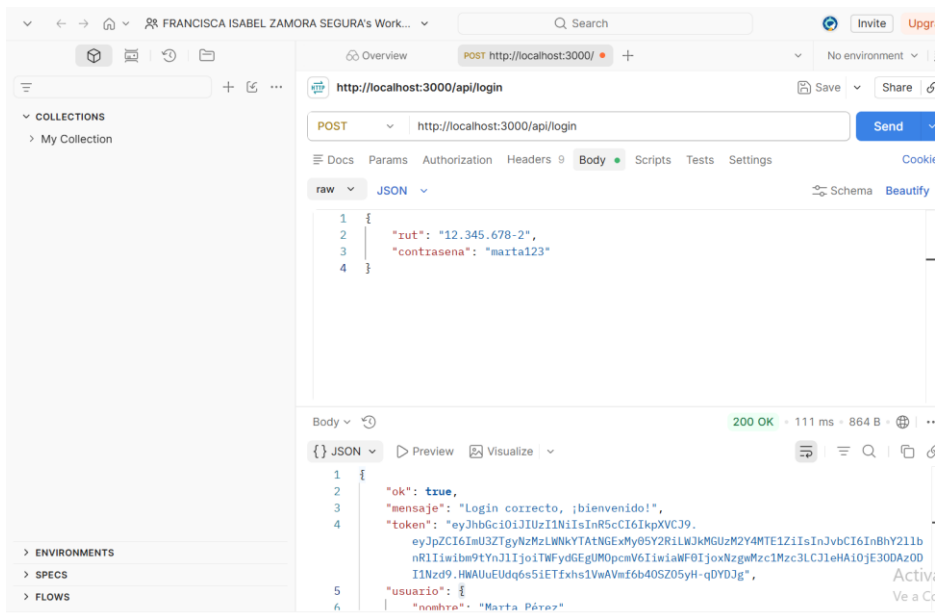
- **Rol Paciente:** Al enviar los parámetros de un usuario de prueba (como el RUT 12.345.678-2, nombre Marta Pérez, correo y contraseña), el servidor responde con un estado HTTP 201 Created y un JSON que confirma la persistencia exitosa del registro, retornando un identificador único.

- **Rol Médico:** Se validó la flexibilidad del endpoint al registrar cuentas para el personal, obteniendo en ambos casos respuestas satisfactorias 201 Created acompañadas de sus respectivos IDs autogenerated.





**Autenticación de Usuarios (POST /api/login):** Se evaluó el comportamiento del endpoint de inicio de sesión enviando las credenciales de acceso correspondientes a un correo y una contraseña válidos. Al coincidir los datos en el servidor, el backend responde con un estado HTTP 200 OK y un objeto JSON que incorpora un token criptográfico firmado. Este token es enviado de vuelta al cliente junto con los datos del perfil del usuario logueado para habilitar la posterior protección de rutas y el manejo seguro de la sesión.



**Inserción de Signos Vitales (POST /api/registro-salud):** Se registran los datos en RegistroSalud, devolviendo un estado HTTP 201 Created:

- **Evaluación de Parámetros Normales:** Al ingresar un control asociado al ID de la paciente Marta Pérez, el backend calcula automáticamente el

campo estado: "Estable" y guarda el documento clínico reflejando la fecha exacta de creación en el servidor.

- **Evaluación de Parámetros Alterados:** Se realizaron pruebas con registros que presentaban sintomatología severa, confirmando que el algoritmo interno del controlador altera correctamente la propiedad a estado: "Crítico" antes de alojar la fila en MySQL.

Las respuestas devueltas en formato JSON demuestran el cumplimiento normativo de la API REST respecto al control de errores, la gestión de esquemas de datos y la consistencia en el almacenamiento relacional del ecosistema de telemedicina.

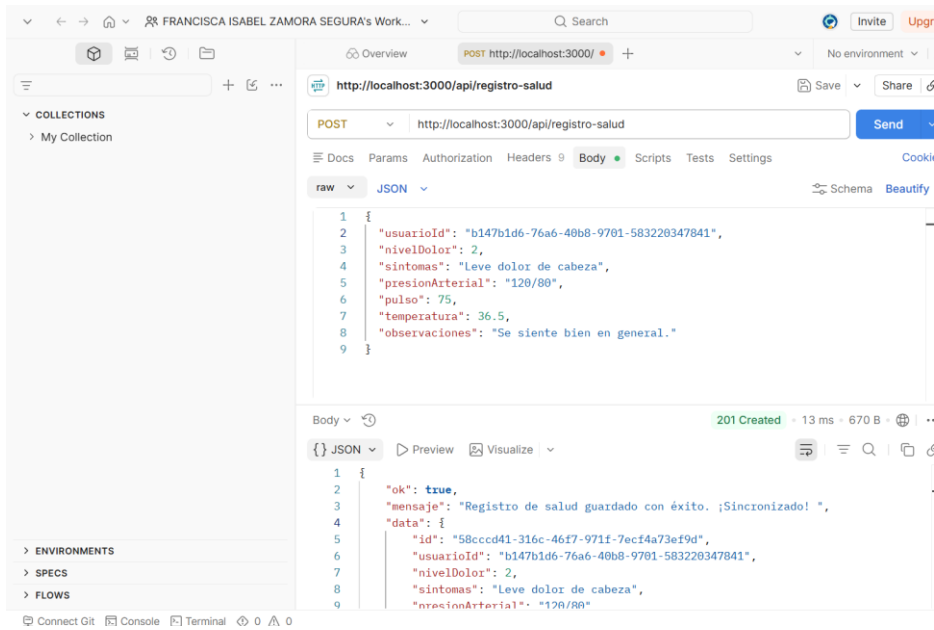
```
1 {
2   "usuarioId": "e7e82733-cda0-4a13-9cdb-bd0e33f8115f",
3   "nivelDolor": 3,
4   "sintomas": "Cansancio generalizado y dolor de cabeza leve",
5   "presionArterial": "120/80",
6   "pulso": 78,
7   "temperatura": 36.5,
8   "observaciones": "Paciente Marta Pérez en reposo clínico."
9 }
```

Body

```
6   "usuarioId": "e7e82733-cda0-4a13-9cdb-bd0e33f8115f",
7   "nivelDolor": 3,
8   "sintomas": "Cansancio generalizado y dolor de cabeza leve",
9   "presionArterial": "120/80",
10  "pulso": 78,
11  "temperatura": 36.5,
12  "observaciones": "Paciente Marta Pérez en reposo clínico.",
13  "estado": "Estable",
14  "fechaCreacion": "2026-06-01T02:54:45.877Z"
15 }
```

```
1 {
2   "ok": true,
3   "mensaje": "Registro de salud guardado con éxito. ¡Sincronizado!",
4   "data": {
5     "id": "196dba5b-8c94-4d60-84ee-ea5be35a75d",
6     "usuarioId": "ef7e86ae-a86e-4161-b575-a6d89f1f79e0",
7     "nivelDolor": 9,
8     "sintomas": "Taquicardia y dolor de pecho fuerte",
9     "presionArterial": "140/95"
10  }
11 }
```





Link GitHub: <https://github.com/fzs701/telemedicina.git>

## Conclusión

Esta primera etapa ha sido clave para sentar las bases de la plataforma de telemedicina, ya que permitió transformar los diseños de Figma en una aplicación real y funcional utilizando Ionic. Lo más relevante del proceso no fue solo la creación de interfaces visuales, sino también la incorporación de seguridad desde las primeras líneas de código mediante rutas protegidas que resguardan la privacidad de los pacientes. Además, se priorizó una experiencia de usuario simple y accesible, permitiendo que cualquier persona pueda registrar su estado de salud sin dificultades técnicas, tanto en computador como en dispositivos móviles. Aunque aún falta la integración con una base de datos real y mejorar aspectos como la mensajería, el sistema cuenta con una base sólida y profesional, preparada para evolucionar en futuras etapas.